CONVERGE

AGENTIC URBAN RESILIENCE PLATFORM

Hackathon Blueprint • Research • Workflows • Architecture • MVP Plan

Working concept document | V1: Smart Cities | V2: Water, Food Security, Healthcare

Quick Brief

Hackathon research, product, architecture, workflows, innovation plan, MVP execution, and team operating guide

> **Working concept. V1 focuses on Smart Cities incident-to-verified-resolution. V2 expands the same orchestration core into Water Management, Food Security, and Healthcare. Treat cross-domain capabilities as a prototype until implemented and tested.**

1. Executive Summary

CONVERGE is a proposed agentic operations platform for turning fragmented civic reports and operational signals into coordinated, evidence-backed, verifiable resolution. The initial product is deliberately narrow: Smart Cities — citizen reports to verified resolution. Its differentiator is not a chatbot, dashboard, map, or AI classifier. It is a stateful response loop that links related incidents, reasons about shared causes, plans actions under real constraints, reacts when plans become stale, and refuses to mark work complete without adequate evidence.

The long-term direction is a shared orchestration engine with domain modules for water networks, food redistribution, and administrative healthcare coordination. The V2 ambition is to show how one disruption can affect multiple services and how coordinated planning can outperform isolated departmental plans.

Hackathon constraint: deliver a credible end-to-end V1 MVP in the first six hours. Only after that loop is reliable should the team attempt the more ambitious innovation layer and a small cross-domain demonstration.

2. Problem Framing and Product Thesis

The problem

Civic issue systems often capture reports and route work, but operational resolution can fail when reports are duplicated, incidents share an upstream cause, departments optimise locally, resources become unavailable, deadlines are missed, or a closure is asserted without contradictory evidence being checked.

Product thesis

A city needs more than a ticketing workflow: it needs an auditable, continuously updated operational picture that can connect events, propose feasible plans, monitor execution, and verify outcomes.

Primary user groups

- Municipal operations teams and department coordinators
- Integrated Command and Control Centre (ICCC) operators
- Field supervisors and repair/work-order teams
- Municipal technology vendors and implementation partners

V1 scope

- Ingest and validate citizen reports and operational events
- Resolve locations/assets and identify duplicates or related incidents
- Build a local incident/dependency graph
- Prioritise and assign work with explicit constraints
- Replan when resources, conditions, or deadlines change

- Validate closure using evidence and reopen/escalate when necessary
- Provide an audit trail and measurable baseline comparison

Non-goals for V1

- Replacing municipal systems of record
- Unsupervised real-world dispatch or public-safety decisions
- Claiming causal certainty from correlation alone
- Building complete water, food, and healthcare products before V1 works
- Presenting simulated outcomes as real-world results

3. Research Plan and Hypotheses

The research sprint should test specific hypotheses rather than collect generic information about smart cities. Keep source-derived facts, research findings, inferred hypotheses, simulated ground truth, and implemented capabilities clearly separated.

ID	Hypothesis	What to investigate / evidence to collect
H1	Recurring incidents may hide a shared upstream cause.	Find cases where multiple local reports map to one asset, outage, blockage, or operational failure. Identify the evidence needed to support or reject the shared-cause hypothesis.
H2	Departments may optimise locally instead of across dependencies.	Look for handoff delays, conflicting work orders, duplicated visits, blocked access, and downstream services affected by a local fix.
H3	A closure status may be mistaken for verified resolution.	Investigate what counts as proof of completion, how contradictory reports are handled, and whether recurrence causes reopening.
H4	Plans become stale when resources or conditions change.	Test crew unavailability, equipment shortages, road closures, weather, deadline changes, and newly discovered higher-priority incidents.
H5	Systems may fail to learn from interventions.	Look for repeat incidents, ineffective fixes, missing post-action checks, and whether recurrence changes future triage or diagnosis.

Research packet checklist

- Official challenge wording and judging criteria, copied exactly from the guidebook
- Existing systems and products: capabilities, limitations, and evidence-backed gaps
- User/workflow map: reporter → triage → department → field team → verification → closure
- Data availability assumptions and privacy constraints
- A testable hypothesis, one measurable outcome, and a go/no-go recommendation

Prior-art guardrail

Do not present AI classification, duplicate detection, dashboards, maps, chat interfaces, or recommendations alone as the core innovation. These are common building blocks. The defensible contribution should be the verified operational loop: evidence-linked incident relationships, constraint-aware replanning, and closure verification, with measured results on reproducible scenarios.

4. Product Scope and Roadmap

V1 — Smart Cities: report to verified resolution

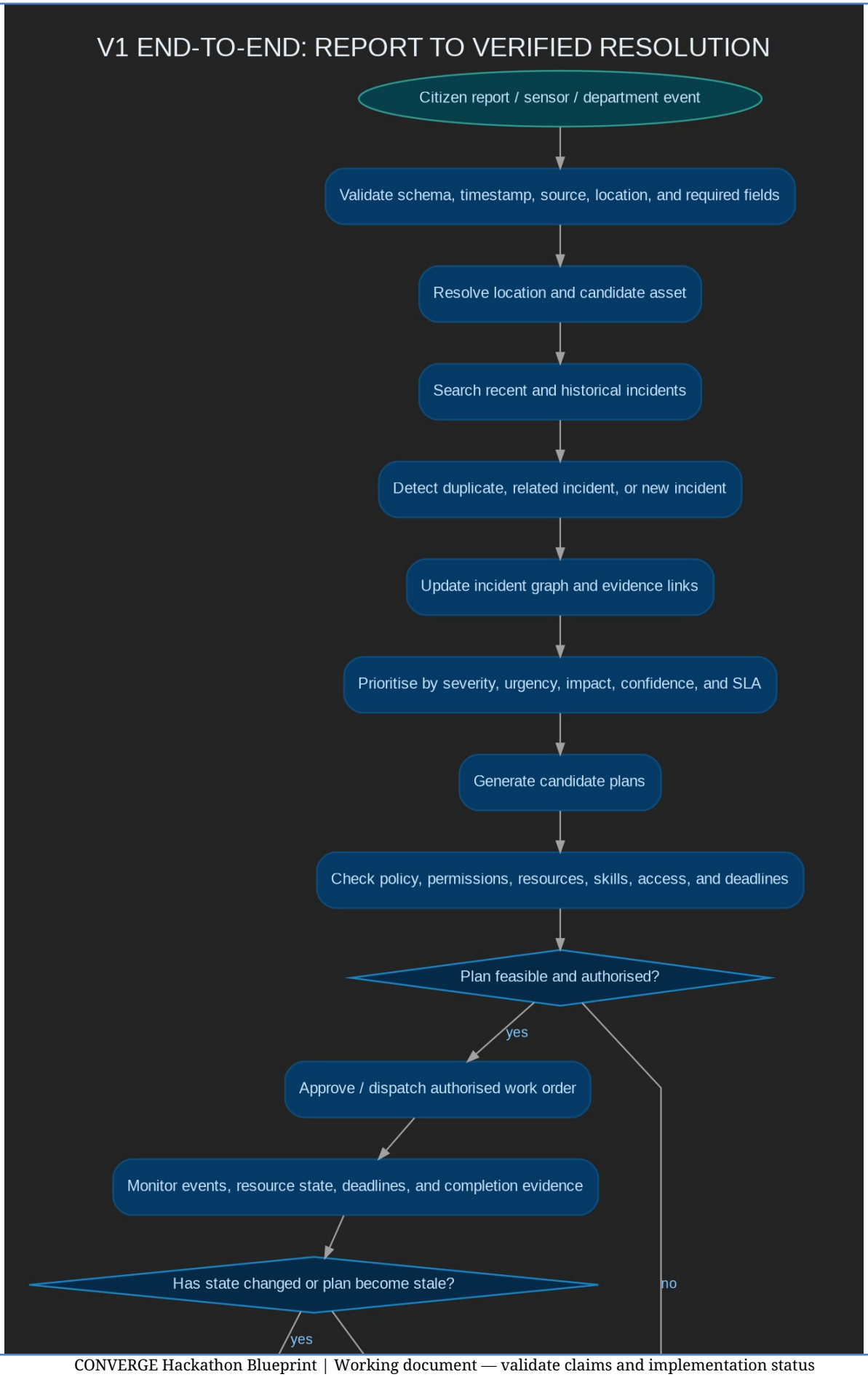
The V1 product handles citizen reports and operational events, groups related incidents, prioritises work, proposes feasible actions, monitors changes, and validates resolution. It should be possible to replay a seeded scenario from a clean state.

V2 — shared core plus domain modules

Module	Responsibilities	Safety boundary
Smart Cities	Incident intake, deduplication, department routing, deadlines, evidence-based closure, recurrence tracking.	Human oversight for consequential dispatch/escalation decisions.
Water Management	Sensor-quality checks, network anomaly detection, fault hypotheses, repair feasibility, post-repair verification.	Do not treat noisy sensor data as ground truth; require authorised repair actions.
Food Security	Supply/demand matching, recipient eligibility, perishability, transport/handling constraints, reallocation after disruptions.	Respect food safety, eligibility, capacity, and human-defined policies.
Healthcare	Administrative pathway tracking, missed-step detection, appointments/referrals, reminders, consent-aware coordination.	Administrative coordination only; no autonomous diagnosis or clinical treatment decisions.

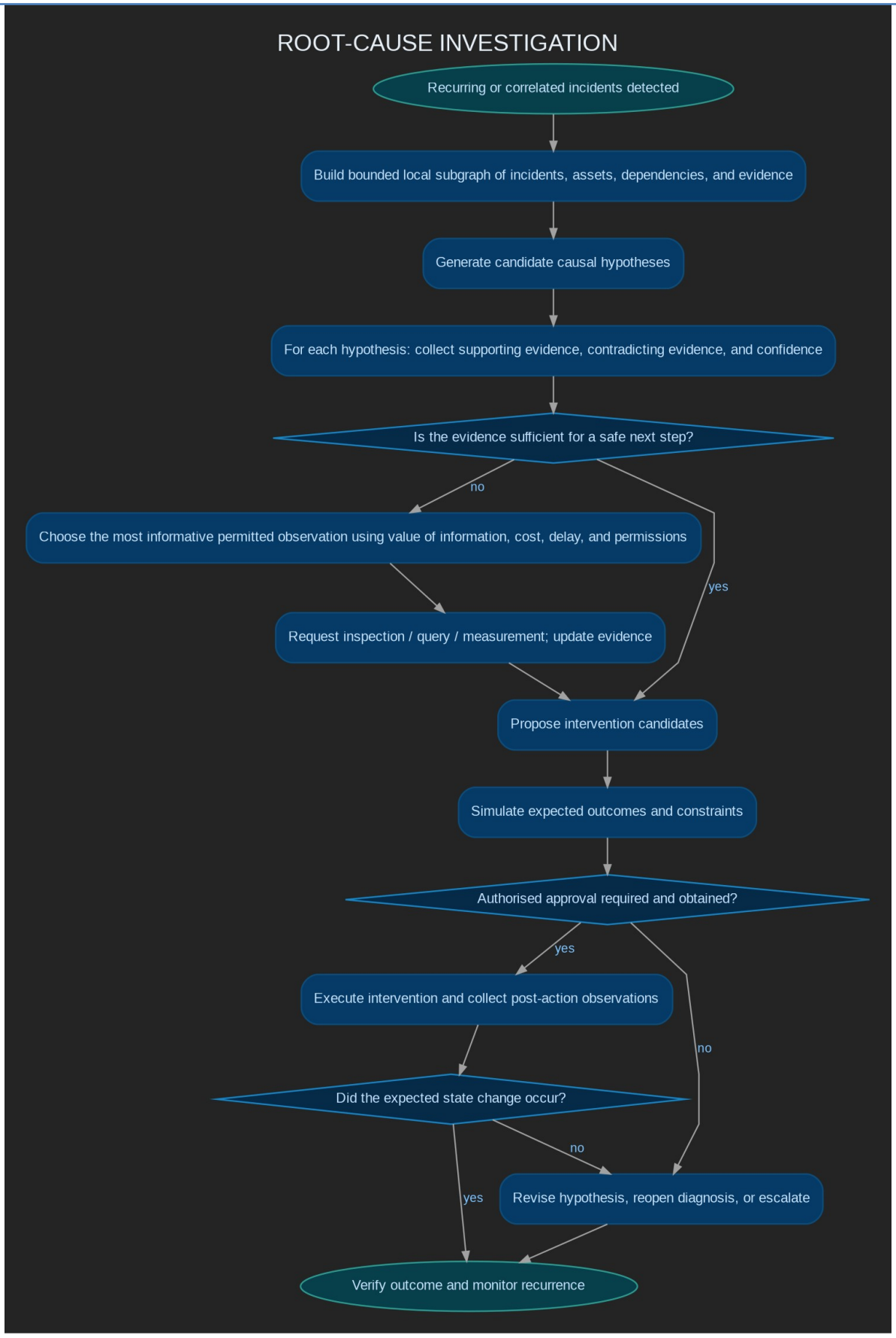
Build the platform as one engine with domain-specific modules, not four separate applications. The shared engine should own event processing, state transitions, policy checks, planning, evidence provenance, audit logs, and evaluation. Each module supplies its own entities, constraints, and tools.

5. End-to-End V1 Workflow


V1 END-TO-END: REPORT TO VERIFIED RESOLUTION
Citizen report / sensor / department event
Validate schema, timestamp, source, location, and required fields
Resolve location and candidate asset
Search recent and historical incidents
Detect duplicate, related incident, or new incident
Update incident graph and evidence links
Prioritise by severity, urgency, impact, confidence, and SLA
Generate candidate plans
Check policy, permissions, resources, skills, access, and deadlines
Plan feasible and authorised?
yes
Approve / dispatch authorised work order
Monitor events, resource state, deadlines, and completion evidence
Has state changed or plan become stale?
yes
no

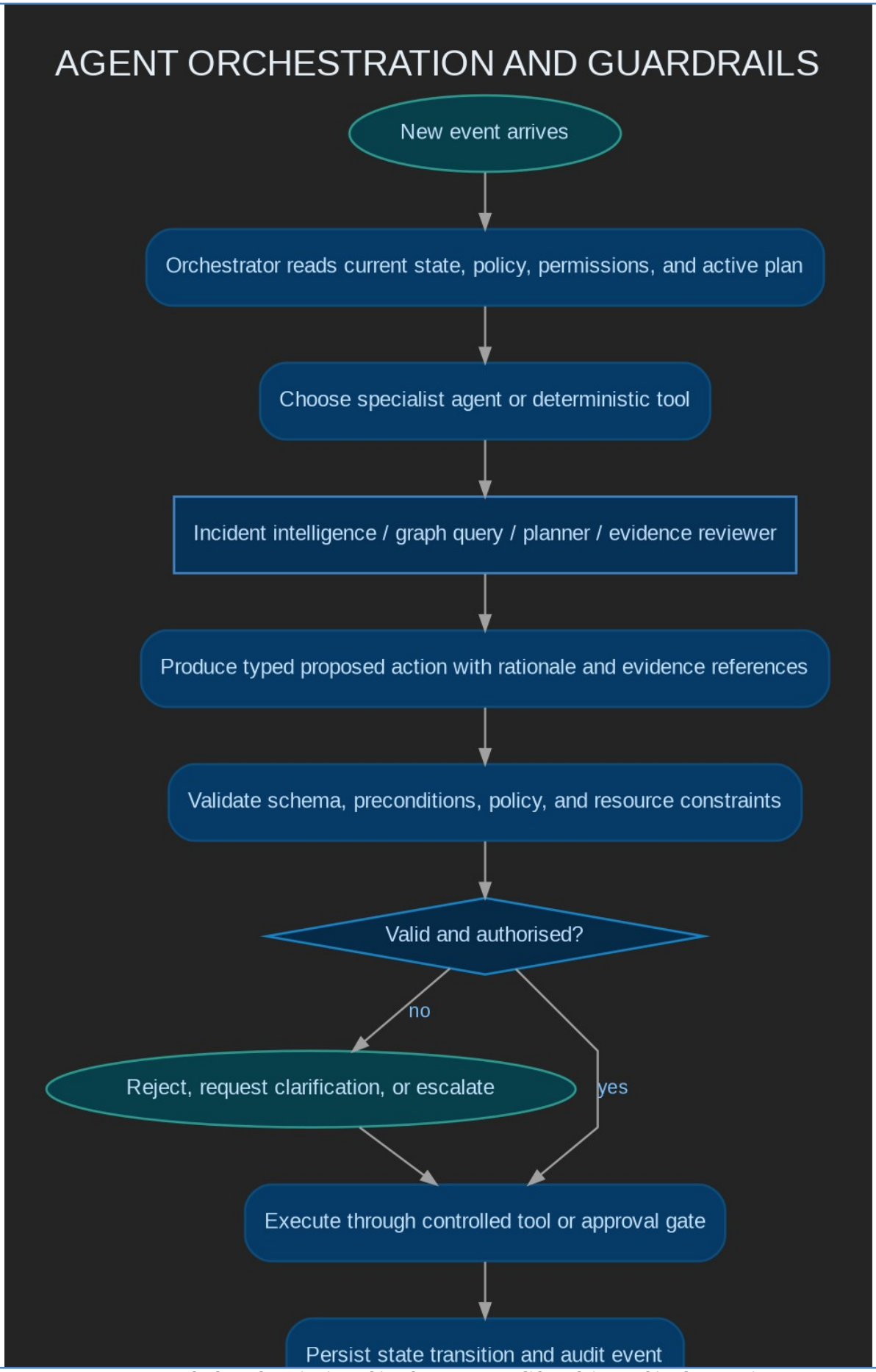
Important invariant: a work order being marked complete is not sufficient evidence that the underlying issue is resolved. V1 should represent the difference between an action being reported as complete and the incident being verified as resolved.

6. Root-Cause Investigation Workflow



Do not claim a model has proven causality. The system should rank hypotheses, show evidence and contradictions, and state uncertainty. Active information gathering is a promising advanced feature if the MVP loop is already stable.

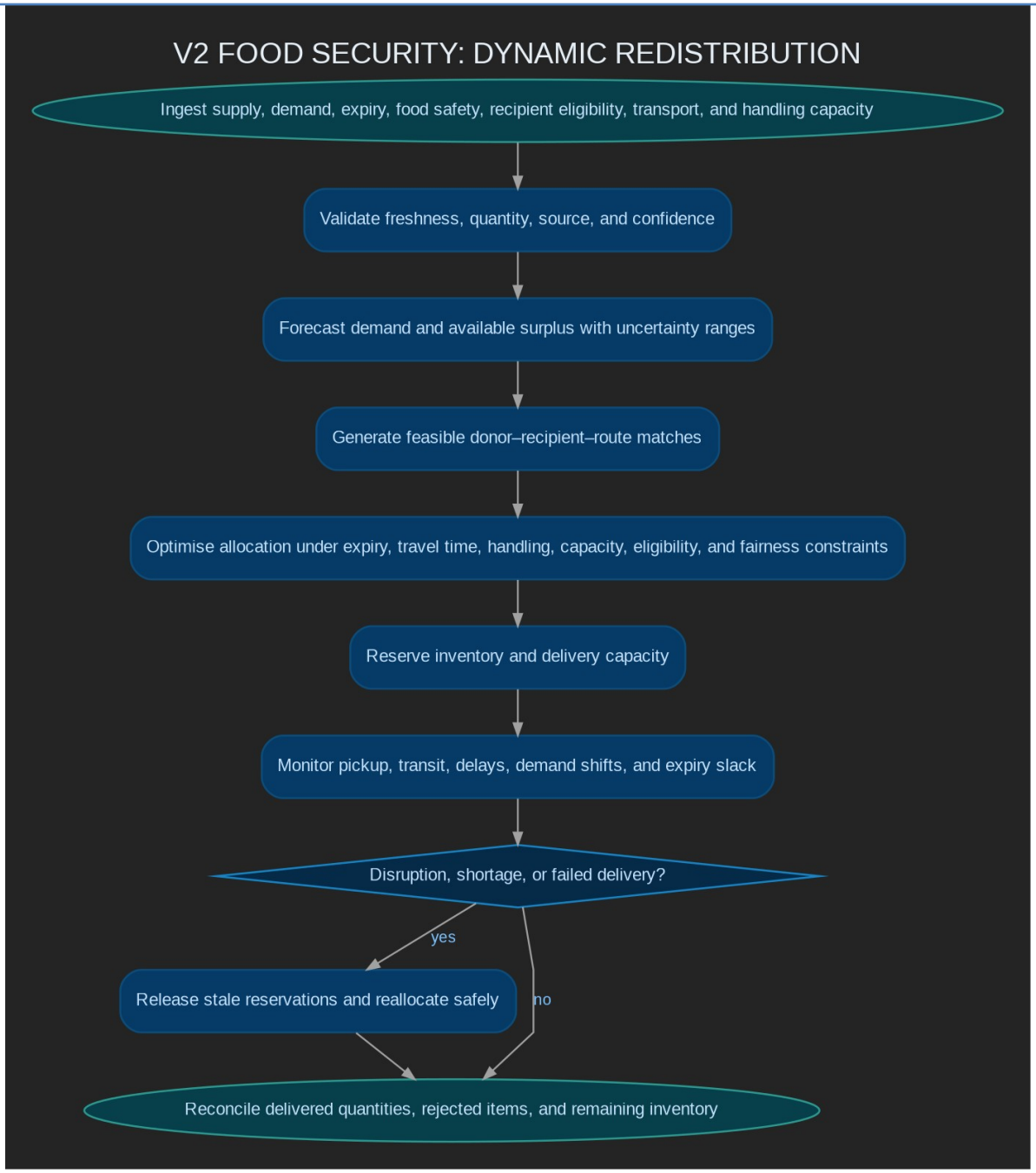
7. Agent Orchestration and Guardrails


AGENT ORCHESTRATION AND GUARDRAILS
New event arrives
Orchestrator reads current state, policy, permissions, and active plan
Choose specialist agent or deterministic tool
Incident intelligence / graph query / planner / evidence reviewer
Produce typed proposed action with rationale and evidence references
Validate schema, preconditions, policy, and resource constraints
Valid and authorised?
no
yes
Reject, request clarification, or escalate
Execute through controlled tool or approval gate
Persist state transition and audit event

Use the LLM for ambiguous interpretation, hypothesis generation, and readable explanations. Use deterministic code for state transitions, hard constraints, permissions, scheduling, and verification rules. Tool outputs should be typed and validated; agents must not directly mutate arbitrary state.

8. V2 Workflows and Difficult Innovations

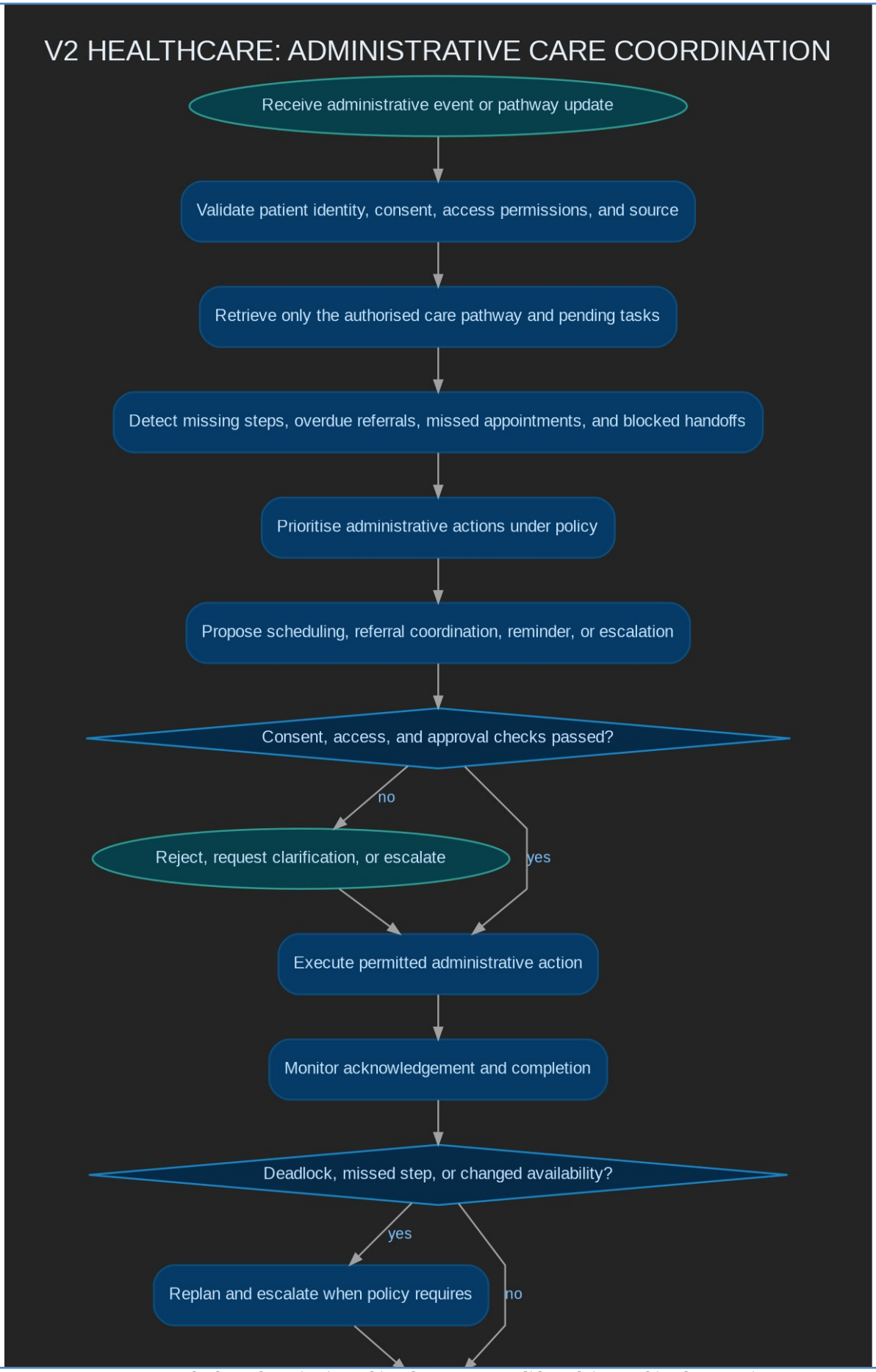
8.1 Food Security — dynamic redistribution



- Perishability-aware allocation: optimise for usable food delivered before expiry, not just nominal quantity moved.

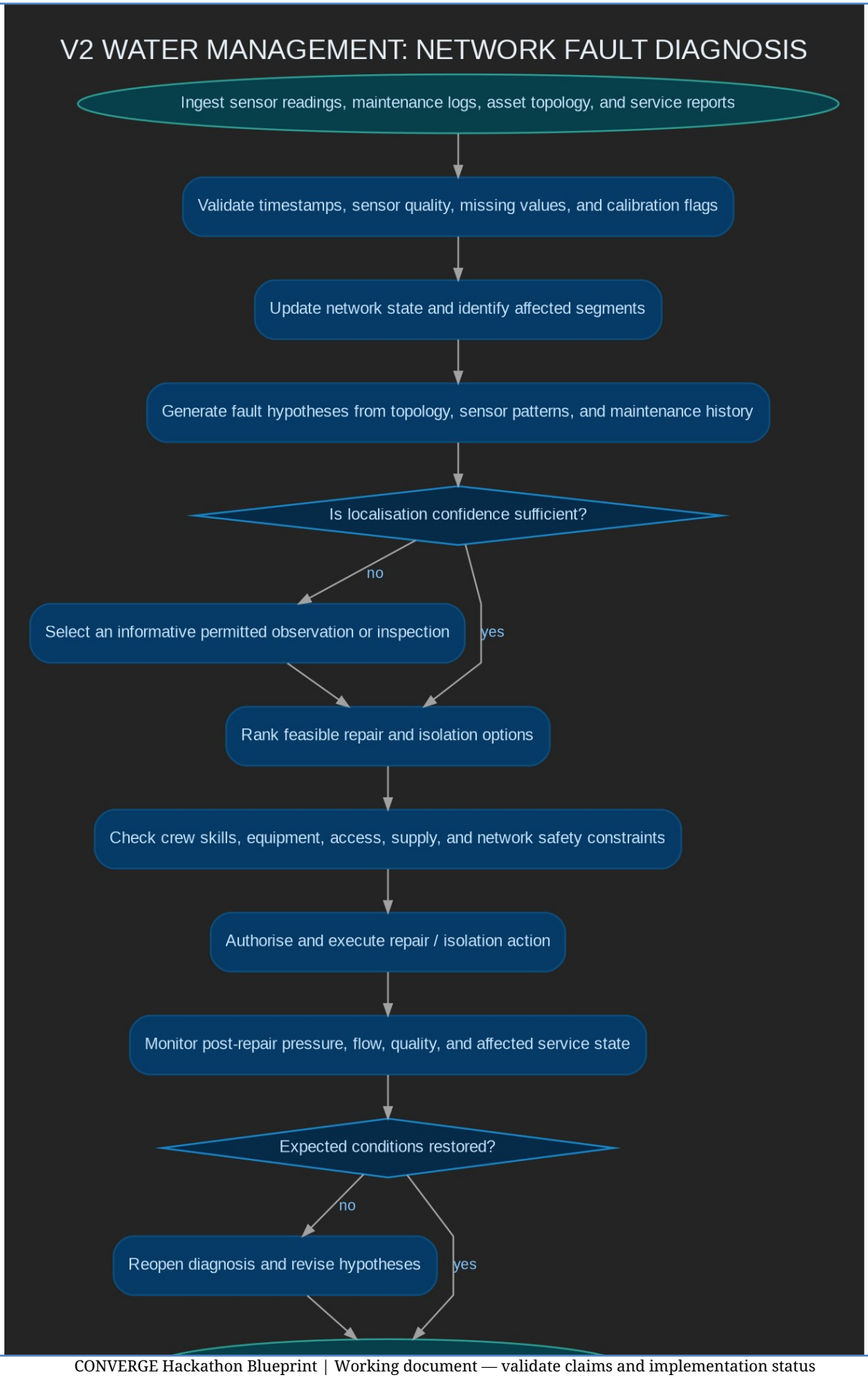
- Opportunity-cost-aware redistribution: consider which feasible recipient match creates the greatest marginal benefit without unfairly starving other recipients.
- Robust planning under uncertain demand and supply: compare plans across plausible scenarios rather than trusting one forecast.
- Delivery recovery: detect delays or failed handoffs early and reallocate before the food becomes unusable.

8.2 Healthcare — administrative care coordination


V2 HEALTHCARE: ADMINISTRATIVE CARE COORDINATION
Receive administrative event or pathway update
Validate patient identity, consent, access permissions, and source
Retrieve only the authorised care pathway and pending tasks
Detect missing steps, overdue referrals, missed appointments, and blocked handoffs
Prioritise administrative actions under policy
Propose scheduling, referral coordination, reminder, or escalation
Consent, access, and approval checks passed?
no
yes
Reject, request clarification, or escalate
Execute permitted administrative action
Monitor acknowledgement and completion
Deadlock, missed step, or changed availability?
yes
no
Replan and escalate when policy requires

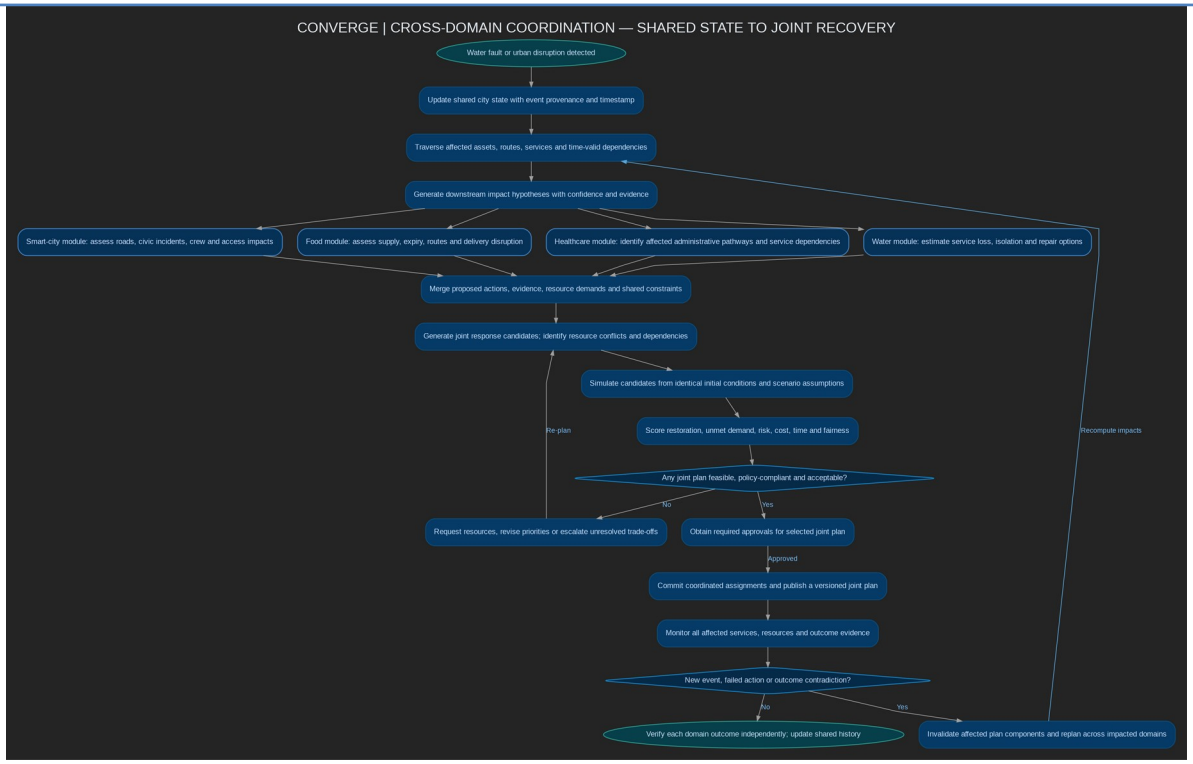
- Care-pathway deadlock detection: find workflows where no next step can progress because a referral, result, appointment, or approval is missing.
- Consent-aware information flow: enforce who may see which information and for what purpose at every handoff.
- Uncertainty-aware escalation: distinguish missing data from confirmed failure and escalate based on risk and policy.
- Auditable administrative autonomy: track every reminder, scheduling attempt, acknowledgement, and exception. No autonomous clinical diagnosis or treatment.

8.3 Water Management — network fault diagnosis


V2 WATER MANAGEMENT: NETWORK FAULT DIAGNOSIS
Ingest sensor readings, maintenance logs, asset topology, and service reports
Validate timestamps, sensor quality, missing values, and calibration flags
Update network state and identify affected segments
Generate fault hypotheses from topology, sensor patterns, and maintenance history
Is localisation confidence sufficient?
no
yes
Select an informative permitted observation or inspection
Rank feasible repair and isolation options
Check crew skills, equipment, access, supply, and network safety constraints
Authorise and execute repair / isolation action
Monitor post-repair pressure, flow, quality, and affected service state
Expected conditions restored?
no
yes
Reopen diagnosis and revise hypotheses

- Active fault localisation: choose the next measurement or inspection that most reduces uncertainty while accounting for cost and delay.
- Topology-aware anomaly interpretation: distinguish local sensor faults from likely network faults using graph structure and neighbouring readings.
- Repair-aware diagnosis: favour hypotheses that explain the evidence and have feasible, safe repair paths.
- Verification-driven maintenance: close only after post-repair observations meet defined restoration conditions.

9. Cross-Domain Coordination Prototype



Difficult cross-domain innovations

- Temporal dependency graph: relationships change over time; store validity windows and evidence, not just static links.
- Multi-objective response optimisation: balance restoration time, unmet demand, risk, cost, fairness, and resource contention.
- Counterfactual response simulation: compare candidate plans under the same initial conditions and explain why one is preferred.
- Cascading-failure recovery: identify downstream services affected by a disruption and coordinate recovery across domain boundaries.

Hackathon-safe demonstration: one water disruption, one affected road or civic incident, one food-delivery dependency, and one healthcare administrative dependency. Show a baseline of independently generated plans, then a coordinated plan. Use seeded data and label results as simulated.

10. Architecture and Shared Data Model

Logical architecture

- Event ingestion API: reports, sensor events, department updates, resource changes, and completion evidence.
- Normalizer and entity resolution: schemas, timestamps, location/asset matching, duplicate candidates.
- State store: incidents, assets, dependencies, resources, actions, plans, evidence, audit events.
- Temporal dependency graph: incidents ↔️ assets ↔️ services ↔️ resources with evidence and validity windows.
- Orchestrator: selects agents/tools, reads state and policy, manages plan lifecycle.
- Policy and constraint engine: permissions, required approvals, hard constraints, SLA rules, safety invariants.
- Planning/optimisation: candidate generation, feasibility checks, scoring, re-planning.
- Evidence and verification service: source provenance, reliability, postconditions, contradictory evidence.
- Evaluation harness: seeded scenarios, baselines, metrics, deterministic replay.
- Frontend: incident queue, graph/timeline, plan comparison, evidence panel, audit trail.

Core entities

Entity	Suggested fields
Incident	id, type, location, severity, status, created_at, deadline, source_ids
Asset	id, type, location, operational_state, metadata
Dependency	source_id, target_id, relationship, valid_from, valid_until, evidence_ref, confidence
Resource	id, type, skills, availability, capacity, location, constraints
Action	id, incident_id, action_type, owner, status, preconditions, evidence_refs
Evidence	id, source, timestamp, content_ref, reliability, verification_status
Plan	id, version, actions, constraints, score, assumptions, status
AuditEvent	id, actor, event_type, timestamp, previous_state, new_state, evidence_refs

Suggested API surface

- POST /events — ingest a report or operational event
- GET /incidents/{id} — retrieve incident state
- GET /incidents/{id}/graph — retrieve related incidents/assets/dependencies
- POST /plans — generate candidate response plans
- POST /plans/{id}/validate — check policy and constraints
- POST /plans/{id}/execute — request authorised execution
- POST /events/simulate — replay a scenario event
- GET /audit/{incident_id} — retrieve state and decision history
- GET /metrics — evaluation metrics

Pragmatic stack

- Frontend: Next.js or React with Tailwind; use a graph/timeline and simple incident queue before polishing maps.
- Backend: FastAPI with Pydantic schemas.

- Storage: SQLite for speed; PostgreSQL/PostGIS only if geospatial queries materially help the MVP.
- Graph: NetworkX for local graph reasoning; avoid infrastructure overhead unless required.
- Optimisation: OR-Tools for constrained scheduling/allocation if the team can demonstrate a meaningful result.
- Water simulation: consider EPANET only if the water prototype genuinely needs hydraulic simulation and time permits.
- LLM: use Claude or the available coding/model tools for interpretation and explanations; keep hard constraints deterministic.
- Testing: pytest, seeded scenarios, replay/reset command, and schema validation.

11. Evaluation: Prove the Mechanism

A polished UI is not proof of innovation. The MVP needs reproducible scenarios and a baseline comparison. Create a small synthetic city with known ground truth and deliberately inject failure cases.

Scenario	Injected condition	Expected system behaviour
Duplicate reports	Several reports describe one underlying incident.	Link reports with evidence; avoid duplicate work orders.
Shared upstream cause	Multiple local issues depend on one asset or disruption.	Raise a shared-cause hypothesis with confidence and supporting/contradicting evidence.
Crew unavailable	Assigned resource becomes unavailable.	Invalidate the old plan and generate a feasible alternative.
Missed SLA	Deadline passes without required progress.	Escalate or replan according to policy; record audit event.
False closure	Work is marked complete but sensor/report evidence contradicts it.	Block closure or reopen the incident.
Verified restoration	Required post-condition evidence arrives.	Close with explicit evidence references and timestamp.
Cross-domain disruption	Water fault affects a delivery route or service.	Coordinated plan should account for downstream dependencies.

Metrics to report

- Time to feasible plan
- Duplicate work orders avoided
- Percentage of closures supported by required evidence
- False-closure detection rate on seeded tests
- Replan latency after resource/condition changes
- SLA breach rate vs. baseline
- Unmet demand / service downtime in cross-domain scenario
- Constraint violations (target: zero in tested cases)
- Audit completeness: percentage of consequential state transitions with traceable evidence

Do not invent results. Run the baseline and CONVERGE on the same seeded scenarios, record actual outputs, and report limitations. If the sample is small, call the results a prototype evaluation rather than a general performance claim.

12. Six-Hour MVP Execution Plan

Time	Goal	Deliverable / exit gate

H0–H1	Freeze scope and contracts.	One-page problem framing, one golden scenario, schemas, API contracts, named owners, acceptance tests.
H1–H2	Build shared core.	Event ingestion, seeded data, incident state, graph links, audit log, reset/replay.
H2–H4	Complete V1 loop.	Report → related incident → priority/assignment → plan change → evidence verification → close/reopen.
H4–H5	Prove the central capability.	Run baseline and system on repeatable scenarios; capture metrics and evidence.
H5–H6	Freeze and stabilise.	Fix critical bugs, test fresh replay, prepare fallback recording/screenshots, rehearse demo.
After H6	Attempt one difficult innovation.	Add active root-cause observation or counterfactual plan comparison; do not destabilise the core.
Final stretch	Show V2 potential.	One small cross-domain disruption prototype; compare independent planning vs coordinated planning; polish and rehearse.

Definition of done for V1

- A report is validated and either linked to an existing incident or creates a new one.
- Related incidents show why they were linked, with evidence references.
- Priority and assignment are explicit and explainable.
- Resource failure or changed conditions trigger state update and replanning.
- Contradictory closure evidence blocks closure or reopens the incident.
- Valid verification evidence can close the incident with a traceable record.
- The entire scenario can be reset and replayed.
- A baseline comparison is recorded.
- Failures do not silently corrupt state or produce unaudited actions.

13. Team and R&D Work Division

Keep R&D focused on decisions the build team needs. Each packet should end with evidence, a recommendation, unresolved questions, and a go/no-go call—not a long unstructured research dump.

Owner / packet	Responsibility	Required output
A — Challenge & workflow	Extract official track wording, judging criteria, user workflow, constraints.	Exact requirements, acceptance criteria, scope risks.
B — Prior art & market	Investigate existing civic reporting, ICCC and work-order products; find evidence-backed gaps.	Competitor matrix, citations, differentiation recommendation.
C — Architecture	Validate event/state model, APIs, agent boundaries, graph, storage and test approach.	Architecture diagram, schemas, dependency risks.
D — Evaluation	Design seeded scenarios, baseline and metrics.	Scenario fixtures, expected outputs, measurement plan.
E — Product & business	Identify buyer, user, deployment path, value proposition and pricing hypotheses.	One-page business case, buyer/user map, assumptions.

Team operating rules

- One person owns the integration branch and demo path.
- Keep changes in isolated branches and merge only tested work.
- Define API/schema contracts before parallel coding.
- Use one canonical seeded dataset and stable IDs.
- Every feature must map to a user problem and a demo acceptance test.
- Freeze scope at H5; after that, prioritise reliability over new features.
- Keep a known-good fallback build and a recorded walkthrough.

14. Business Model and Adoption Hypotheses

Potential buyers include municipal corporations, ICCC operators, and municipal technology vendors. These are hypotheses to validate through research and stakeholder conversations, not confirmed market demand.

- Enterprise subscription priced by city/site, department, or operational volume.
- Implementation and integration fees for existing complaint systems, GIS, sensors, and work-order platforms.
- Advanced analytics and reliability reporting for recurrence, SLA performance, and service restoration.
- Pilot strategy: begin with one bounded civic workflow and a synthetic/demo integration; expand only after measured operational value.

Value proposition to test

Can CONVERGE reduce duplicate work, shorten time to a feasible response, improve evidence-backed closure, and reduce stale plans when conditions change? Build the demo around these measurable claims rather than claiming to automate an entire city.

15. Risk Register and Mitigations

Risk	Why it matters	Mitigation
Scope explosion	Four domains can become four unfinished products.	One shared core; V1 Smart Cities only; V2 is one small proof-of-concept.
LLM hallucination	Invented causes or actions undermine trust.	Evidence-linked hypotheses, explicit uncertainty, typed outputs, deterministic validation.
Unsafe autonomy	Real-world dispatch or healthcare actions can have consequences.	Human approval gates, policy checks, limited simulation, least privilege.
Bad or missing data	Incorrect locations, timestamps, and sensor readings distort decisions.	Validate schemas and data quality; expose uncertainty; request informative observations.
Fake verification	A completion status can be mistaken for a resolved issue.	Post-condition checks, contradictory evidence handling, explicit closure criteria.
Stale plans	Resources and conditions can change after planning.	Version plans, detect invalidated assumptions, replan on meaningful events.
Demo failure	New features may break the core loop.	Freeze at H5, reset/replay tests, fallback recording, deterministic seeded scenarios.
Unsupported novelty claims	Generic features are already common.	Research prior art; claim only what the prototype implements and measures.

Core design principle

Distinguish facts, inferred hypotheses, simulated ground truth, and implemented/verified capabilities. The system should show what it knows, why it believes it, what remains uncertain, and what evidence would change the decision.

16. Demo Storyboard

- 1. Submit three citizen reports that appear different but share a hidden asset or incident.
- 2. Show the system linking them and exposing the evidence behind the relationship.
- 3. Generate candidate actions and show hard constraints or unavailable resources.
- 4. Change the world state (e.g., crew unavailable or route blocked) and show the old plan becoming stale.
- 5. Replan and explain what changed.
- 6. Mark a work order complete while injecting contradictory evidence; show closure being blocked or incident reopened.
- 7. Supply valid post-condition evidence; show verified closure and a complete audit trail.
- 8. Optional V2 proof: introduce one water disruption that affects a road and food/health administrative dependency; compare independent and coordinated plans.
- 9. Finish with measured baseline results and honest limitations.

A strong demo tells one coherent story from event to verified outcome. Avoid spending the presentation on architecture jargon before the audience has seen the failure mode and the operational difference.

17. Immediate Next Actions

1. Confirm the official primary track and copy its exact problem statement into the project README.
2. Pick one golden V1 scenario and define its expected state transitions.
3. Create typed schemas and API contracts before parallel implementation.
4. Assign owners A–E for research packets and set a short report deadline.
5. Build the seeded event → incident → plan → verification loop first.
6. Add one difficult innovation only after the baseline is repeatable.
7. Record all evaluation outputs and label simulated data clearly.
8. Prepare a stable fallback and rehearse the demo against a clean reset.

Appendix A — Minimum Golden Scenario

A sample synthetic scenario for the first end-to-end test: three reports arrive from nearby locations within a short time window; two are duplicates and one is a related symptom. A candidate shared asset is implicated but not proven. The planner assigns an authorised crew. Before dispatch, the crew becomes unavailable. The system invalidates the plan and proposes an alternative. A work order is then marked complete, but a follow-up report contradicts the claim. Closure is blocked and the incident reopens. Finally, a valid post-condition observation arrives, the system verifies restoration, closes the incident, and preserves the full evidence/audit trail.

This scenario is intentionally synthetic. Its purpose is to validate the workflow and invariants, not to claim real municipal performance.

Appendix B — Terms

- Incident graph: a bounded network of incidents, assets, resources, services, and evidence links.
- Temporal dependency: a relationship that may be valid only during a specific time window.
- Value of information: the expected benefit of an observation in reducing decision uncertainty relative to its cost and delay.
- Plan invalidation: marking a proposed plan stale because a precondition or assumption changed.
- Verified closure: closing an incident only after defined post-conditions are supported by adequate evidence.
- Counterfactual comparison: comparing candidate plans from the same initial state under explicit assumptions.